

RECURRENT

재귀(RNN) 추론 — 메모리 캐싱 최적화 검증

핵심: 선형 어텐션 계열(linear/dla/titans)은 학습은 병렬($O(L^2)$), **추론은 고정 크기 상태를 캐싱하는 RNN 재귀($O(1)$ 상태)**로 풀 수 있다. 두 경로는 수학적으로 동치다. 설계 §8(엣지 고정상태 추론, 길이 무관 평평한 메모리) · §3.4(동치성 테스트)의 실측 근거.

검증 환경: RTX 4090 24GB, torch 2.10.0+cu128. 재현: `verify_recurrent.py`.

이론

병렬: $D[t,s] = \prod_{j=s+1..t} a_j$, $o_t = (\phi(q_t) \cdot \sum_s D[t,s] \phi(k_s) \otimes v_s) / (\phi(q_t) \cdot \sum_s D[t,s] \phi(k_s))$
재귀: $S_t = a_t \cdot S_{t-1} + \phi(k_t) \otimes v_t$, $z_t = a_t \cdot z_{t-1} + \phi(k_t)$, $o_t = (\phi(q_t) \cdot S_t) / (\phi(q_t) \cdot z_t)$

$S_t = \sum_{s \leq t} D[t,s] \cdot \phi(k_s) \otimes v_s$ 로 전개되어 분자·분모 모두 병렬형과 일치(코드 감사로 대수 확인). → 재귀는 $L \times L$ 어텐션 행렬을 만들지 않는다(메모리 캐싱 최적화). 상태 S, z 는 $O(dh^2)$ 로 길이 무관 상수.

1. 동치성 (병렬 vs 재귀 출력 $\max|diff|$) — 병렬 에이전트 검증

base_rule	equiv diff	stress worst diff*	판정	상태 캐시(B1)
linear	3.6e-07	8.3e-07	✅ PASS	135,168 B
dla (감쇠)	3.9e-07	8.1e-07	✅ PASS	135,168 B
titans (영속메모리)	3.6e-07	7.8e-07	✅ PASS	135,168 B
swla	0 (윈도우 캐시, RNN 아님)	—	✅ (병렬 위임)	0

* stress = aggregation{residual,grm,soup,ssc} × segmentation{constant,logarithmic} × batch4 × L320 전 조합 최악값. 모두 합격 기준 $1e-3$ 대비 ~1,200배 여유(fp32 노이즈 수준). dla의 데이터 의존 감쇠, titans의 영속 메모리가 있어도 동치 유지 확인.

2. 메모리 캐싱 이점 (길이별 peak GPU 메모리, dla, batch1)

L	병렬 $O(L^2)$	재귀 $O(L)$ +상수상태
512	101.5 MB	70.8 MB
1024	206.1 MB	77.1 MB
2048	618.3 MB	89.7 MB
4096	2254.2 MB	114.9 MB (~20×↓)
8192	8772.5 MB	(상태 135KB 고정)
16384	OOM	동작 가능

병렬은 L^2 로 폭증 → 16k에서 **OOM**. 재귀는 $L \times L$ 행렬이 없어 완만히 증가, 상태 캐시는 **135KB로 길이 무관 고정**. 옛지에서 임의 길이 입력을 상수 메모리로 처리 가능.

⚠ **정정 (COMPARE3.md)**: 위 "병렬"은 *naive 구현*($L \times L$ materialize, $O(L^2)$)이라 16384 OOM이었다. 이는 구현 결함 → **chunked(GLA 표준)로 재구현해 $O(L)$ 해결**(16384=321MB, OOM 없음, FlashAttn 트랜스포머 371MB보다도 적음). 정확도는 chunked=naive=재귀 동치($5e-7$). 추론 메모리 이점은 그대로: 트랜스포머 KV캐시 $O(L)$ (16384=128MB) vs 재귀 상태 $O(1)$ (132KB) $\approx$ **992×**.

3. 적대적 코드 감사 결과

- 대수 전개 sound(분자/분모 모두 $D[t,s]$ 전개와 일치), titans 영속메모리·RMSNorm·aggregation·checkpoint init 양쪽 동일 적용 확인.
- 수정한 버그(B1)**: 병렬 `linear` 의 `torch.ones(...)` 에 dtype 미지정 → fp32에선 무해하나 혼합정밀도(bf16/fp16)에서 재귀형과 dtype 불일치 가능. `dtype=x.dtype` 지정으로 수정.
- 한계(설계): `init_mode=checkpoint` 는 전체 시퀀스 평균을 쓰므로 **고정길이 입력 가정**(진정한 토큰 스트리밍에는 부적합). `forward_recurrent` 는 추론 전용(@no_grad).

4. 학습 파이프라인 통합 (상수메모리 eval)

`trainer.RealRun._eval` 이 `eval_recurrent=True` (기본)면 추론을 ****재귀 경로**(`forward_recurrent`)**로 수행 → 평가도 옛지 배포와 동일한 상수메모리 경로. 동일 평가 배치(고정 seed)에서:

- recurrent eval recall == parallel eval recall (**완전 일치**), CE diff **0.0**. → 평가 품질 손실 없이 메모리 경로만 교체. swla는 내부에서 병렬로 위임.

5. 진정한 토큰 스트리밍 API (step 단위, $O(1)$ /스텝)

`GrowingMemoryModel.step(token, t, states)` / `.stream(x)` — 토큰 1개씩 처리하며 고정 상태만 갱신. 옛지 실시간 추론용. 검증(병렬 forward vs 토큰 스트리밍, $L=80$):

base_rule	stream vs parallel diff	판정
linear	4.2e-07	✓
dla	4.2e-07	✓
titans	4.2e-07	✓
swla	4.2e-07	✓ (윈도우 링버퍼 O(W) 캐시)

- rnn(linear/dla/titans): 상태 S,z만 유지 → 스텝당 O(1), 시퀀스 길이 무관.
- swla: 마지막 W개 (k,v) 링버퍼 → O(W) bounded 캐시.
- 제약: `init_mode=checkpoint` 는 전체 컨텍스트 평균이 필요해 스트리밍 불가 → `independent` 만 지원.
- 재현: `python3 verify_recurrent.py --rule <rule> --stream --json`

검증 방법

- 병렬 에이전트 4개 동시 실행: base_rule별(linear/dla/titans) 4090 stress 검증 3개 + 재귀식↔병렬식 대수 동치 코드 감사 1개. 메모리 벤치는 GPU 경합으로 수치가 오염되므로 단독 실행.
- 재현: `python3 verify_recurrent.py --rule <linear|dla|titans> --stress --bench`